# SwiftLog Logistics: Intelligent Logistics Prediction & Supply Chain Optimization

**Technical Report | Week 5 | Machine Learning Track (Mission ML05)**

Prepared by Gyimah Ramsey Opoku, Junior Machine Learning Engineer, AnalystLab Africa Consulting. SwiftLog Logistics Limited is a fictional client and all data is synthetic.

## Executive Summary

SwiftLog asked for machine learning models that predict delayed deliveries, product returns, demand and customer value. I prepared the eleven datasets, engineered leakage-safe features and built four models on four different logistics problems, with five algorithms each, cross-validation, hyper-parameter tuning, feature selection and class-imbalance handling, and compared every one against a naive baseline.

**The main finding is negative, and it is well supported.** No model predicts better than the baseline: delivery delay reaches a test ROC-AUC of 0.52 (95% interval 0.48 to 0.56), product returns 0.49 (0.42 to 0.56), and the two regression models have R-squared of -0.06 (demand) and -0.003 (customer value). Correlations, 1 significant results out of 22 hypothesis tests (about one is expected by chance), mutual information and label-shuffled models all agree: in these tables, delay, returns, demand and customer value behave like independent random draws.

The value delivered is therefore a reusable and tested ML workflow, a quantified statement of what SwiftLog's current data can and cannot support, a data-collection roadmap, twelve evidence-based recommendations, and a working demo application. Two issues were caught that would have produced falsely impressive results: a duplicate of the delay column in Delivery_Performance, and the absence of any supplier link.

## 1. Problem Definition

SwiftLog manages orders, inventory, suppliers, warehouses, shipments, employees and returns across several regions. Management wants to move from descriptive reporting to proactive decisions. The ML objectives were: predict delayed deliveries before they occur, identify orders likely to be returned, forecast product demand, and estimate customer value. Success was defined before modelling: a model must beat a naive baseline on held-out data, with an uncertainty interval, to count as useful.

| Problem | Type | Target | Datasets |
|---|---|---|---|
| Delivery delay | Classification | Shipment 3+ days late | Orders, Shipments, Order_Items, Products, Customers, Warehouses, Inventory, Employees |
| Product return | Classification | Order has a return record | Orders, Order_Items, Customers, Returns (+ Shipments) |
| Demand forecasting | Regression | Units demanded next month | Orders, Order_Items, Products, Inventory |
| Customer value | Regression | Spend in months 6 to 9 | Customers, Orders, Order_Items, Returns |
| Supplier reliability | Not built | n/a | No table links suppliers to any shipment or product |

## 2. Data Preparation

Eleven tables were imported and audited (no missing cells, no duplicate rows, every foreign key resolves). Dates were parsed, text trimmed and cleaned copies written to Processed_Datasets. Key findings from the audit:

- **Target leakage caught.** Delivery_Performance.DelayDays is identical to Shipments.DelayDays in 100% of rows, and its Status is exactly "DelayDays of 3 or more". Both are used only to define the target, never as features. A model given either would score about 100% for a meaningless reason.
- **No supplier link.** No table other than Suppliers contains a SupplierID, so Supplier Reliability Prediction is excluded rather than invented.
- **Orders without items.** 236 orders have no line items; they were kept with a has_items flag and zero basket features.
- **Implausible records.** About 9% of cancelled orders have returns, a sign the fields were generated independently.
- **Partial month.** September 2026 ends on 8 September, so it is excluded from monthly totals and trends.
- **Missing values and encoding.** The only gaps are history features for a customer's first order, imputed (median) inside the pipeline. Categories are one-hot encoded and numbers standardised inside a scikit-learn Pipeline so nothing is learned from test data.
- **Splits.** Classification: 70/15/15 stratified train/validation/test (random_state 42). Forecasting: time-based (train months 3 to 6, validation month 7, test month 8). Customer value: random customer split.

## 3. Feature Engineering

One row per order (5,000 rows), enriched from six tables. History features use only earlier orders.

| Group | Features | Why it was created |
|---|---|---|
| Basket / order size | lines, quantity, value, average and max price, category shares | Larger or pricier orders may be harder to ship |
| Seasonality | month, weekday, weekend, day of month; monthly demand trend | Seasonal congestion and demand |
| Customer history | prior orders, prior delay rate, prior return rate | Past behaviour as a predictor (earlier orders only) |
| Warehouse capacity and load | headcount, total stock, low-stock share, shipments per employee | Utilisation and congestion proxies |
| Demand panel | lag 1, lag 2, historical mean and spread, trend, price, stock | Standard forecasting inputs |

Excluded on purpose: DelayDays and delay Status (they define the target), order status (settled after delivery) and return history for the delay model (happens after delivery). The final table is saved as Feature_Engineered_Dataset.csv.

## 4. Exploratory Analysis

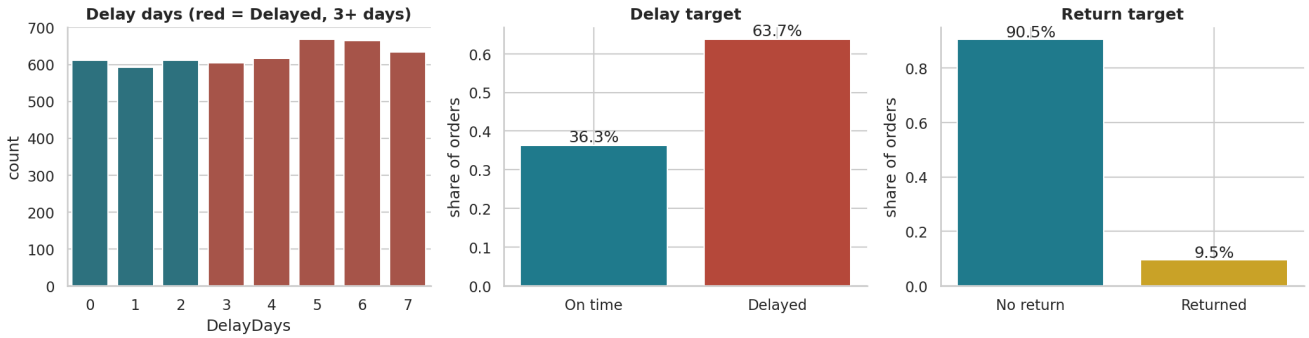



Figure 1. Target distributions. Delay is mildly imbalanced (64/36); returns are strongly imbalanced (9.5% positive).

Class imbalance handling: class weights (balanced / scale_pos_weight) and ROC-AUC, recall and F1 instead of accuracy for returns, because always predicting "no return" is already about 90% accurate. Outliers by the 1.5 x IQR rule are only 0.4% to 2.4% of values and are genuine large orders, so they were kept. The correlation heatmap and the 22 hypothesis tests agree: the strongest correlation of any feature with either target is about 3.7%, and only one test was significant at 5%.

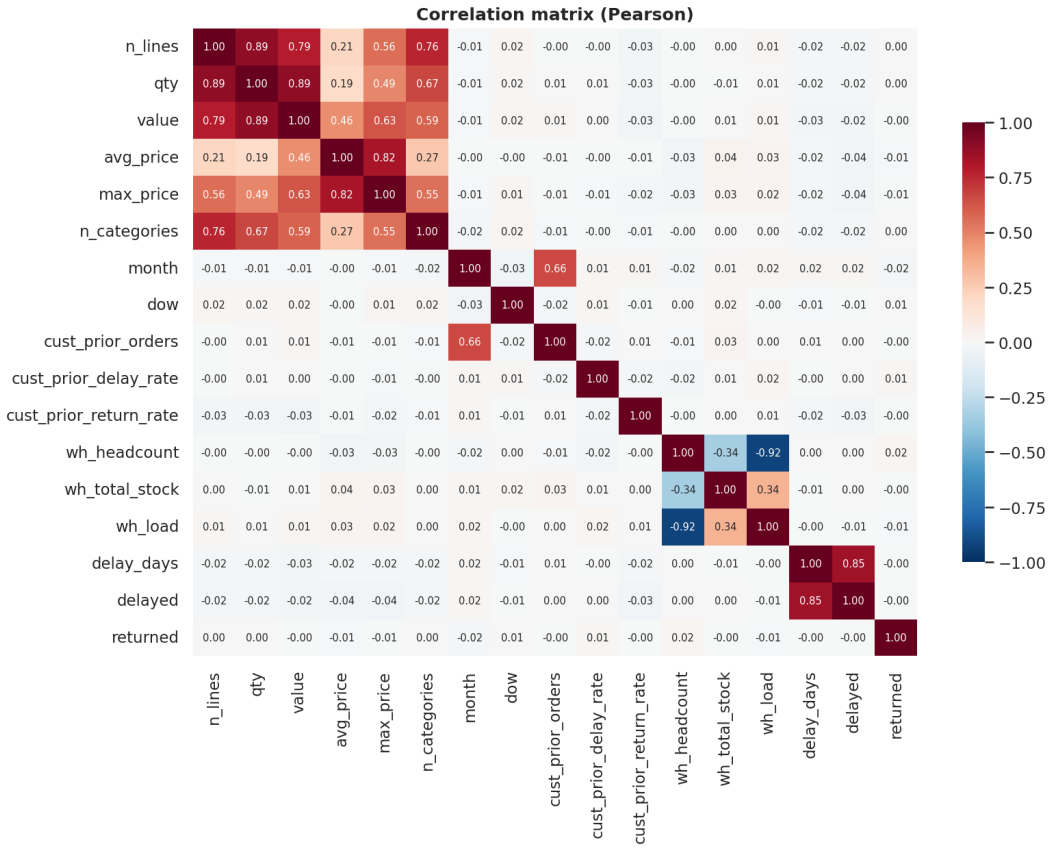



Figure 2. Correlation matrix. Apart from obvious links (value with quantity and price), nothing relates to the targets.

## 5. Model Development

Every problem was run through the same protocol so the comparison is fair: a baseline, five algorithms with default settings (regularised linear model, decision tree, random forest, gradient boosting, XGBoost for classification; linear, ridge, random forest, gradient boosting, XGBoost for regression), 5-fold cross-validation, then tuned versions (GridSearchCV for the linear models with feature selection via SelectKBest, RandomizedSearchCV for the tree ensembles). Selection used the **one-standard-error rule**: among models whose cross-validated score is within one standard error of the best, choose the simplest. When models are statistically tied this avoids rewarding noise with a complicated model.

| Problem | Selected model | Tuned settings |
|---|---|---|
| Delivery delay | Logistic Regression (tuned) | C = 0.1, top 5 features (SelectKBest) |
| Product return | Logistic Regression (tuned) | C = 1, top 5 features |
| Demand forecasting | Ridge Regression (tuned) | alpha = 1000, top 5 features |
| Customer value | Ridge Regression (tuned) | alpha = 1000, top 5 features |

Note what tuning did: for three of the four problems it pushed the model toward heavy regularisation (very strong penalties, five features), which is what an optimiser does when there is no signal: it moves toward predicting the average.

## 6. Model Evaluation

| Problem | CV score (train) | Test result | Other test metrics | Baseline |
|---|---|---|---|---|
| Delivery delay | AUC 0.518 +/- 0.025 | AUC 0.520 (0.48 to 0.56) | Acc 0.55 \| Prec 0.64 \| Rec 0.66 \| F1 0.65 | AUC 0.50 |
| Product return | AUC 0.499 +/- 0.041 | AUC 0.491 (0.42 to 0.56) | Acc 0.47 \| Prec 0.09 \| Rec 0.51 \| F1 0.15 | AUC 0.50 |
| Demand forecast | RMSE 13.85 +/- 0.57 | RMSE 14.75 \| R2 -0.058 | MAE 11.30 \| MSE 218 | Mean: RMSE 14.79 \| R2 -0.064 |
| Customer value | RMSE 6135 +/- 469 | RMSE 6320 \| R2 -0.003 | MAE 5118 \| MSE 3.99e+07 | Mean: RMSE 6313 \| R2 -0.000 |

The full comparison of all nine candidates per problem is in Models/Model_Comparison.xlsx. Two checks make the result trustworthy rather than merely disappointing: bootstrap 95% intervals on test AUC, and a label-shuffle test in which the same pipeline trained on randomly permuted labels scores AUC 0.50 and 0.49, the same as the real models.

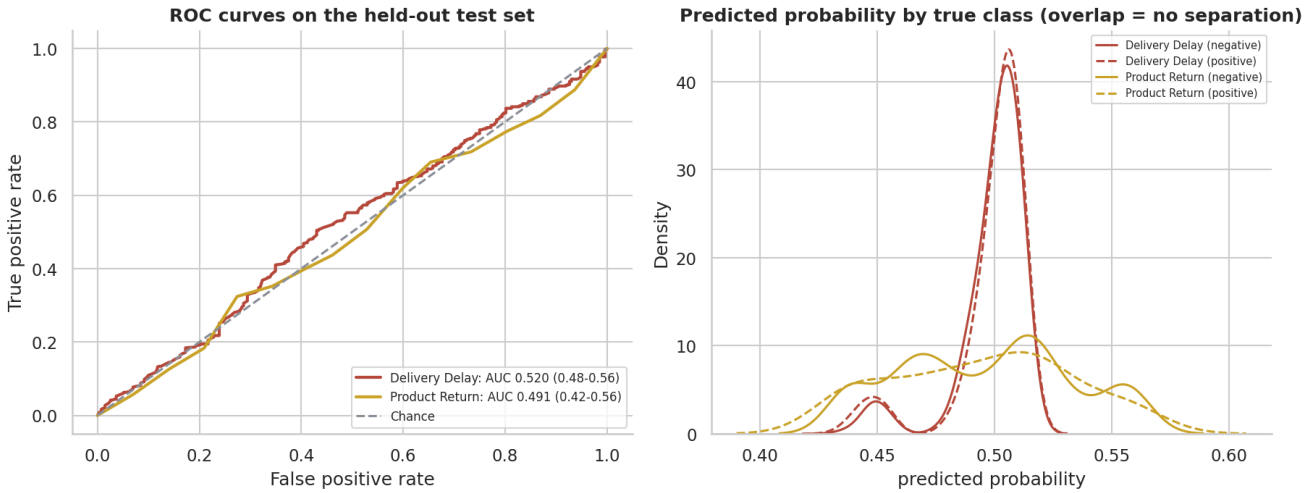



Figure 3. ROC curves hug the diagonal, and predicted probabilities for delayed and on-time orders overlap completely.

The demand model's test RMSE (14.75) is 0.04 below the mean-prediction baseline (14.79) with R-squared still negative: a difference well inside the cross-validation spread, not a real gain. The "last month" forecast is far worse (RMSE 20.5), because month-to-month demand is uncorrelated.

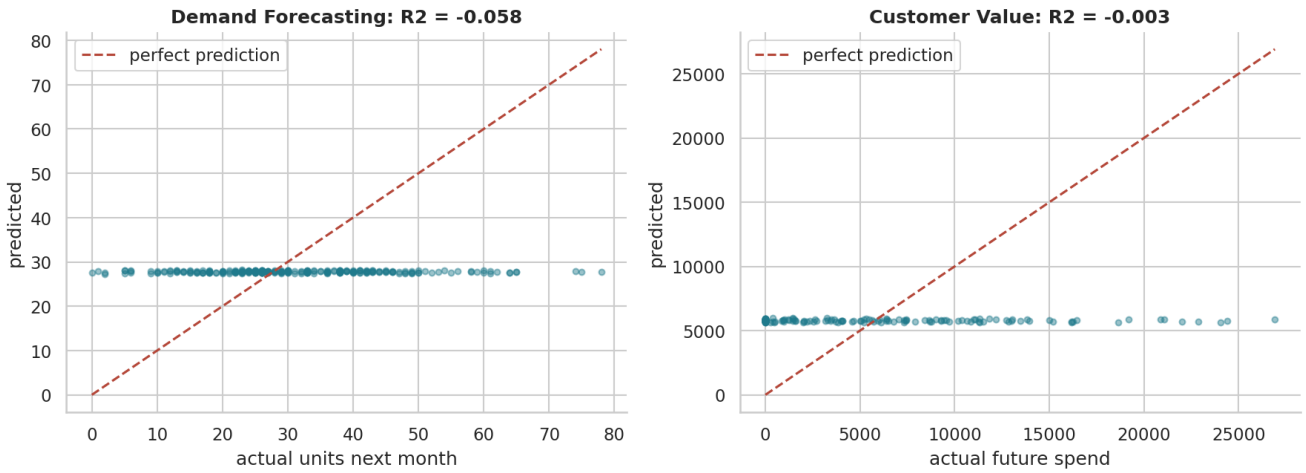



Figure 4. Regression models: predictions are nearly flat, showing there is no pattern to fit.

## 7. Model Interpretation

Three views were used: permutation importance on the test set (what the model relies on for unseen data), SHAP values (how features move individual predictions) and error analysis.

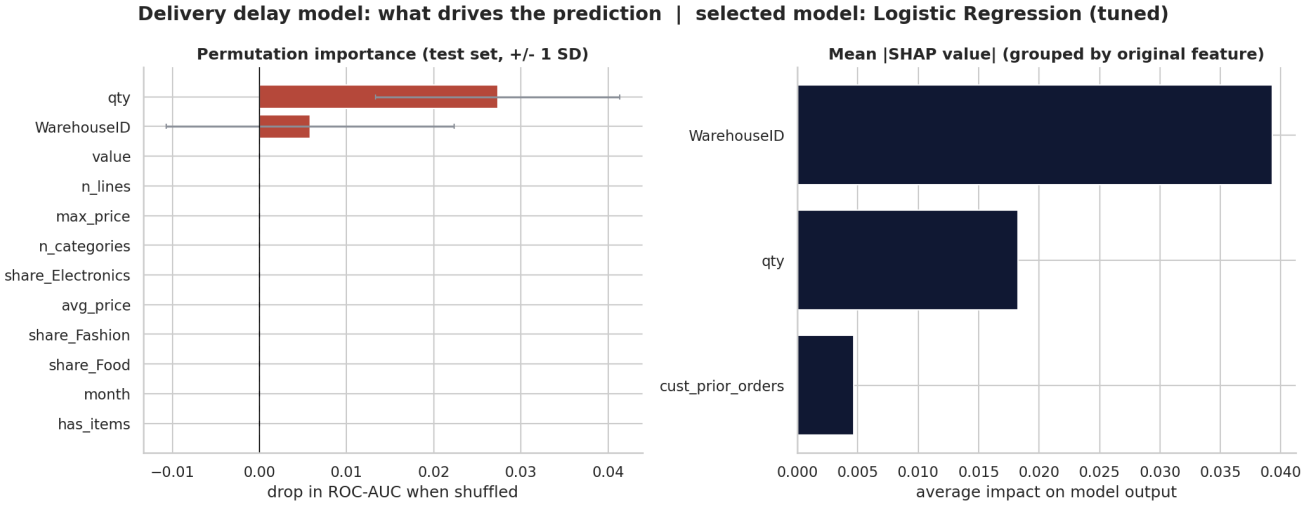



Figure 5. Delay model. Permutation importance (left) is near zero with intervals that include zero; SHAP (right) describes what the model fitted, not what drives delays.

- The delay model keeps only five of its features after selection. Its SHAP ranking (warehouse, quantity, prior orders) reflects fitted noise: permutation importance on new data does not confirm it.
- The return model's top permutation feature (warehouse stock per shipment) and the customer model's (early units) have no business logic behind them and do not generalise; the demand model's lag features contribute about 0.0005.
- Error analysis: the delay model's predicted probabilities all lie between about 0.43 and 0.53, so nearly every prediction is low-confidence; its recall of 0.66 and precision of 0.64 simply mirror the 64% base rate. For returns, precision of 0.09 equals the base rate of 9.5%, so flagged orders are no more likely to be returned than any other.
- Limitations: eight months of data, no timestamps within a day, no carrier, route, supplier or stock-movement fields, a single snapshot of inventory, and synthetic generation that appears to make fields independent.
- How it supports decisions: its main decision-support value is the negative result. It tells management, with evidence, that the current fields cannot drive delay or return prediction, and which fields to collect

next.

## 8. Business Insights

- **Customer value is evenly spread.** Non-cancelled revenue by city: Kano 3.87M, Lagos 3.65M, Ibadan 3.50M, Abuja 3.42M. The top 10% of customers account for 23.9% of revenue.
- **Revenue by product.** The top 20% of products produce 36.0% of revenue (close to proportional). Fashion 37.5%, Food 32.8%, Electronics 29.7%.
- **Delivery.** 63.7% of shipments are delayed (average 3.6 days). Warehouses range from 58.7% to 68.0%, but the difference is not statistically significant (p = 0.33). Lagos destinations are the lowest at 61.0%, Kano the highest at 65.7% (p = 0.10, also not significant).
- **Returns.** 9.5% of orders. Reasons: Late Delivery 185, Wrong Item 163, Damaged 152. Return rates are the same across categories (9.6%, 9.4%, 9.4%) and product-level differences are consistent with chance (p = 0.87).
- **Inventory.** 1,007,764 units in stock against about 6,736 per month demanded; median product covers 150 months. 380 of 4000 stock lines are under 50 units and 9 are at zero.
- **Staffing.** Shipments per employee range from 20 to 38 across warehouses.
- **Most suitable variables for future modelling:** none of the current fields. The strongest candidates once collected: timestamps, carrier, route distance, stock movements, supplier ID and customer attributes.

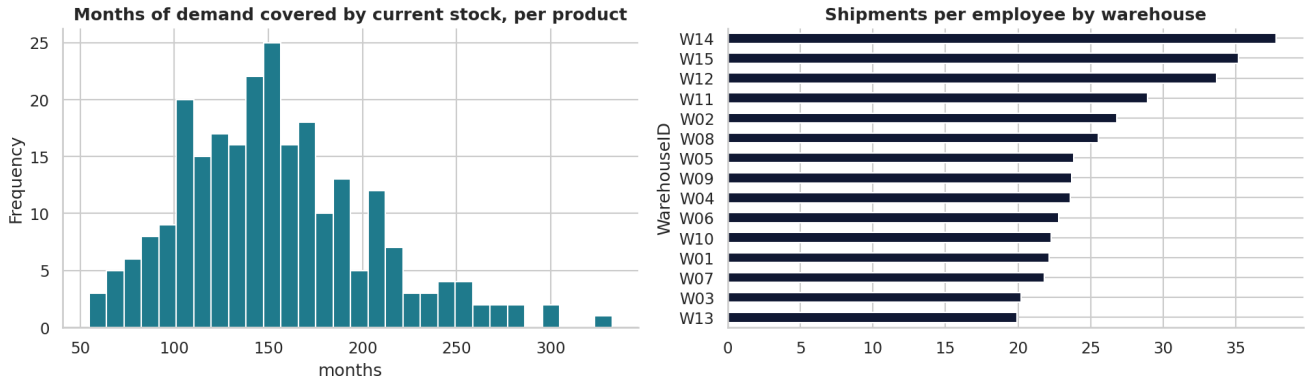



Figure 6. Inventory cover and warehouse workload.

## 9. Recommendations

**1. Do not use the delay or return predictions for operational decisions yet.** Test ROC-AUC is 0.52 (delay) and 0.49 (returns); both intervals include 0.50. Acting on these scores would mean acting on noise. Set a release bar (for example, AUC above 0.65 on a held-out month) before any model drives routing or staffing.

**2. Treat the 64% delay rate as a process problem, not a warehouse problem.** 63.7% of shipments are 3+ days late and only 12.2% arrive on the promised day, yet delay rates are statistically the same across all 15 warehouses (p = 0.33). A uniform failure points to something systemic such as dispatch scheduling, carriers or the delay definition, so start with a root-cause review, not site rankings.

**3. Start recording the data that could actually explain delays.** Add dispatch and pickup timestamps, carrier, route and distance, vehicle, weather and public-holiday flags. Without them the models have nothing to learn from, which is exactly what the near-zero importances show.

**4. Add a SupplierID to products, inventory and shipments.** No table links suppliers to anything, so Supplier Reliability Prediction could not be built. A single foreign key, plus order and receipt dates, unlocks

the whole supplier use case.

**5. Review inventory levels: stock far exceeds demand.** Current stock is 1,007,764 units against roughly 6,736 units demanded per month, a median of 150 months of cover per product. Confirm the order data is complete (it may be a sample); if it is, working capital is tied up in excess stock.

**6. Fix the data-quality gaps found in the audit.** 236 orders have no line items, 9.2% of cancelled orders carry returns, the data dictionary only says "Dataset" for every file, and Delivery_Performance duplicates Shipments. Add validation checks at source so these cannot recur.

**7. Tackle the root causes of returns, not individual products.** Returns are 9.5% of orders. Wrong item (33%) and damaged (30%) together account for about 63% of reasons, which are picking and packaging errors; late delivery adds 37%. Product return rates do not differ from chance ($p = 0.87$), so blacklisting products is not supported.

**8. Rebalance warehouse staffing against workload.** Shipments handled per employee range from 20 to 38 across warehouses, almost a twofold spread. Delay is not yet linked to this in the data, but the imbalance is a cost and fairness issue worth a pilot.

**9. Run controlled pilots to generate cause-and-effect evidence.** Where observational data has no signal, experiments create it: for example, try an alternative dispatch rule or carrier on a random share of orders and compare delay. This also gives future models real variation to learn from.

**10. Spread customer retention effort broadly.** Revenue is not concentrated: the top 10% of customers make up 23.9% of revenue and the four cities are within about 13% of each other (Kano is highest). A broad service-quality programme will pay off more than a VIP scheme, until richer customer attributes support real segmentation.

**11. Collect 24+ months of dated history for demand forecasting.** Eight months of flat demand (lag-1 correlation about 0.02) cannot support seasonality or trend. Dated stock movements and at least two full years are needed before forecasting can add value over a simple average.

**12. Govern the ML workflow now so it is ready when the data improves.** Keep the leakage-safe pipeline, model registry, monitoring and a re-evaluation rule (champion versus challenger on fresh data) in place. The notebook can be re-run unchanged when new fields arrive.

**Future AI opportunities.** Route and dispatch optimisation once route data exists; text classification of return and complaint reasons; anomaly detection on shipment times; and a retrieval-based assistant over operational data (the Generative AI track's work) to give managers answers in plain language.

## 10. Conclusion

The work met its technical goals: a clean, leakage-safe pipeline, four models across two task types, thorough evaluation and explainability, and a working application. Its honest conclusion is that SwiftLog's current data does not support predicting delays, returns, demand or customer value (test AUC 0.52 and 0.49; R-squared near zero). Reporting that clearly, and showing management exactly which data to collect, is more useful than a model that looks impressive for the wrong reasons. Because the whole workflow is re-runnable, the same notebook can be applied to richer data the day it exists.

---

Deliverables: Data/, Notebook/Week5_MachineLearning.ipynb, Models/ (Best_Model.pkl, Model_Comparison.xlsx, Feature_Importance.png), App/ (Streamlit app), Reports/, Presentation/, README.md, References/.